

CONTROL DE VERSIONES CON GIT Y GITHUB

En el desarrollo de software, un **sistema de control de versiones** (VCS, por sus siglas en inglés) funciona una bitácora detallada para el código fuente. En un entorno de trabajo en equipo, resuelve un problema fundamental: el caos de la colaboración concurrente. Sin un VCS, los desarrolladores suelen sobrescribir el trabajo de sus compañeros de manera accidental, pierden versiones funcionales del sistema o terminan con multitud de archivos que fungen como una versión final.

El control de versiones permite que múltiples usuarios editen el mismo proyecto al mismo tiempo en sus máquinas locales, rastreando hizo qué cambio, en qué momento y por qué, lo que permite auditar el código o revertir errores sin perder el progreso de los demás.

Es muy común confundir Git con GitHub, pero no son sinónimos; tienen una relación estructuralmente similar a la que existe entre el motor de un automóvil y una red de carreteras. **Git** es la tecnología subyacente: un software de control de versiones distribuido que se instala de forma local en la computadora. Funciona en la terminal y gestiona el historial de los archivos mediante algoritmos. Por su parte, **GitHub** es una plataforma en la nube (un servicio de alojamiento de terceros) que guarda y sincroniza repositorios de Git en internet. GitHub facilita la colaboración remota ofreciendo una interfaz gráfica y herramientas sociales para equipos, pero depende totalmente de Git para existir, mientras que Git puede funcionar perfectamente sin GitHub.

Todo el ecosistema nace en el **repositorio**, que es esencialmente el directorio raíz del proyecto donde Git inicializa su base de datos para rastrear el historial; por ejemplo, la carpeta que contiene todo el código y los recursos de una nueva aplicación. Al explorar este directorio, el punto de partida visual suele ser el **archivo README**, un documento de texto que sirve como la carta de presentación del proyecto, explicando para qué sirve el software, cómo compilarlo y cómo ejecutarlo. A la par, existe el **archivo .gitignore**, un archivo de configuración que le indica a Git qué elementos locales debe ignorar deliberadamente y jamás subir a la nube. Un ejemplo clásico de esto es configurarlo para ignorar los archivos ejecutables u objetos compilados (como los .o o .exe en C++), ya que estos pueden generarse en cualquier máquina y solo ensucian el historial ocupando espacio innecesario.

El registro del progreso dentro del repositorio se hace mediante **confirmaciones**, conocidas universalmente como **commits**. Un commit es una "imagen" inmutable del estado de tus archivos en un momento específico, acompañada de un mensaje descriptivo. Por ejemplo, si logras optimizar un ciclo de búsqueda, creas un commit para guardar ese hito en la historia. Sin embargo, para no alterar el código principal que ya es estable, se utilizan las **ramas (branches)**. Una rama es una línea de tiempo alternativa e independiente dentro del mismo repositorio. Mientras la rama principal conserva el software que está en producción, puedes crear una rama llamada prueba-nuevo-algoritmo para experimentar sin miedo a romper el sistema.

Una vez que terminas tu trabajo en esa rama aislada, necesitas integrar tus avances al código principal mediante una **fusión (merge)**. El proceso de merge toma los commits de tu rama y los combina con la principal. No obstante, si tú y un compañero modificaron exactamente la misma línea del mismo archivo en ramas distintas, Git no sabrá con qué versión quedarse, produciendo un **conflicto de fusión**. Git pausará el proceso y requerirá intervención humana para que un desarrollador decida qué porción de código preservar y cuál descartar. Para que todo este proceso de integración sea ordenado en equipos grandes, se utiliza un **Pull Request** (o solicitud de extracción). Es un mecanismo formal en plataformas como GitHub donde un desarrollador avisa: "He terminado mi trabajo en esta rama, por favor, revisen mis cambios antes de autorizar la fusión con la rama principal".

Esto nos lleva directamente al flujo de trabajo basado en ramas (Branching Workflow), una metodología estándar en la ingeniería moderna. En este flujo, la regla inquebrantable es que la rama principal (usualmente llamada main o master) debe ser siempre estable y desplegable. Cualquier desarrollo de una nueva característica o corrección de un error se realiza obligatoriamente en una rama paralela. El código solo se integra a la rama principal tras pasar por una rigurosa revisión entre pares (peer review) a través del Pull Request. Al tener a otros ingenieros analizando el código propuesto, se detectan errores de lógica que el autor pasó por alto, se asegura que el código sea eficiente y mantenible, y se fomenta la transferencia activa de conocimiento dentro del equipo antes de que una mala implementación se convierta en deuda técnica.